

TASK 3: SECURE CODING REVIEW

Introduction

Secure coding is an important part of software development because it helps protect applications from security threats and vulnerabilities. In this task, a simple Python login application was reviewed to identify common security issues and improve the overall security of the code.

Objective

The objective of this task was to analyze a Python application, identify potential security vulnerabilities, and apply secure coding practices to make the application safer and more reliable.

Programming Language and Application

Programming Language: Python

Application Reviewed: Simple Login System

The selected application allows a user to enter a username and password to access the system. Although it is a basic application, it contains common security issues that are often found in real-world software.

Original Code

```
username = input("Enter Username: ")
password = input("Enter Password: ")

if username == "Vaibhav_jha" and password == "Vaibhav005":
    print("Login Successful")
else:
    print("Invalid Credentials")
```

Security Vulnerabilities Identified

1. Hardcoded Credentials

The username and password are directly written in the source code. If someone gains access to the code, sensitive information can be exposed easily.

2. Plain Text Password Storage

The password is stored and compared in plain text, which is not secure. Attackers can easily read the password if the code is exposed.

3. No Input Validation

The application accepts user input without validation. This can lead to unexpected behavior and security risks.

4. No Protection Against Brute Force Attacks

The application allows unlimited login attempts, making it vulnerable to brute-force attacks.

Review Method

The application was reviewed manually to inspect the source code and identify common security weaknesses. The review focused on authentication, password handling, and input management.

Improved Secure Code

```
import hashlib

stored_username = "Vaibhav_jha"
stored_password = hashlib.sha256("Vaibhav005".encode()).hexdigest()

username = input("Enter Username: ")
password = input("Enter Password: ")

hashed_password = hashlib.sha256(password.encode()).hexdigest()

if username == stored_username and hashed_password == stored_password:
    print("Login Successful")
else:
    print("Invalid Credentials")
```

Sample Output

```
Enter Username: Vaibhav_jha
Enter Password: Vaibhav005
Login Successful
```

Security Improvements Implemented

1. Passwords are hashed before comparison.
2. Sensitive information is handled more securely.
3. Authentication process is improved.
4. The risk of exposing plain text passwords is reduced.

Recommendations and Best Practices

- Never store passwords in plain text.
- Use password hashing techniques such as SHA-256 or bcrypt.

- Avoid hardcoding sensitive credentials in source code.
- Validate and sanitize all user inputs.
- Limit failed login attempts to prevent brute-force attacks.
- Keep software and dependencies updated.
- Perform regular security reviews and testing.

Remediation Steps

- Replace plain text passwords with hashed passwords.
- Store credentials securely using environment variables or databases.
- Add input validation mechanisms.
- Implement account lockout after multiple failed login attempts.
- Conduct periodic security assessments.

Conclusion

This secure coding review helped identify several common security weaknesses in a simple login application. After analyzing the code, improvements such as password hashing and better authentication practices were implemented. Through this task, I gained practical knowledge of secure coding principles and learned how small security improvements can significantly enhance the safety of an application.